

Implementation of a spatial and spatio-temporal query portal on GTFS data

Mustafa Utku KALAY, Muntaha Fadlelmola, Mariam Sherif Abdelfattah Ibrahim
Bilgisayar Mühendisliği Bölümü
Yıldız Teknik Üniversitesi, 34220 İstanbul, Türkiye
ukalay@yildiz.edu.tr, {muntaha.fadlelmola, mariam.ibrahim}@std.yildiz.edu.tr

Özetçe —Bu çalışma, Genel Toplu Taşıma Besleme Spesifikasyonu verilerine dayalı mekansal ve mekansal-zamansal bir sorgu portalının uygulanmasını sunmaktadır. İstanbul için tarifeli toplu taşıma verileri bir PostgreSQL veritabanına aktarıldı ve durak ve güzergah bilgileri PostGIS kullanılarak mekansal nesnelere dönüştürüldü. Ardışık durak-zaman kayıtlarından bir toplu taşıma ağı oluşturuldu ve Dijkstra, A* ve Gezgin Satıcı Problemi (TSP) gibi rota optimizasyonu sorguları pgRouting ile çözüldü. Sistem, kullanıcıların durakları, güzergahları, en kısa yolları, yoğun durakları ve araç hareketlerini görselleştirmelerini sağlayan bir FastAPI arka ucu ve web tabanlı bir harita arayüzü sunmaktadır.

Anahtar Kelimeler—GTFS, PostgreSQL, PostGIS, pgRouting, MobilityDB, Mekansal Sorgulama, Toplu Taşıma

Abstract—This study presents the implementation of a spatial and spatio-temporal query portal based on General Transit Feed Specification data. Scheduled public transportation data for İstanbul was imported into a PostgreSQL database, and stop and route information were converted into spatial objects using PostGIS. A transit network was created from consecutive stop-time records, and routing queries such as Dijkstra, A*, and Traveling Salesman Problem (TSP) route optimization were implemented with pgRouting. The system provides a FastAPI backend and a web-based map interface that lets users visualize stops, routes, shortest paths, busy stops, and vehicle movements.

Keywords—GTFS, PostgreSQL, PostGIS, pgRouting, MobilityDB, Spatial Query, Public Transportation

I. INTRODUCTION

Public transportation systems generate large amounts of structured data, including stops, routes, trips, timetables, and vehicle movements. The General Transit Feed Specification, or GTFS, is commonly used to represent scheduled transit information in a standard format. The GTFS dataset used in this project was obtained from the IETT open data source [1]. However, raw GTFS files are not directly suitable for map-based querying or route analysis. To make this data useful, it must be imported into a database, cleaned, converted into a spatial format, and integrated with routing algorithms.

This project aims to build a GTFS-based spatial and spatio-temporal query portal. The first phase focuses on scheduled GTFS data because a suitable real-time GTFS source for İstanbul was not available during development. The system stores GTFS files in PostgreSQL, adds spatial support through PostGIS, builds a transit graph for routing,

and exposes the results through FastAPI endpoints and a web interface.

II. SYSTEM ARCHITECTURE

The system consists of four main layers. This layered structure follows the general principles of system design, in which the system is divided into connected components to make implementation and maintenance easier [2]. The first layer is the GTFS dataset, which includes files such as agency, routes, stops, trips, stop_times, shapes, and calendar. The second layer is the PostgreSQL database, where these files are stored as relational tables. PostGIS is used to store stops as point geometries and route shapes as line geometries. The third layer is the query and algorithm layer, where pgRouting and MobilityDB are used for routing and trajectory-based operations. The final layer is the FastAPI backend and web map interface.

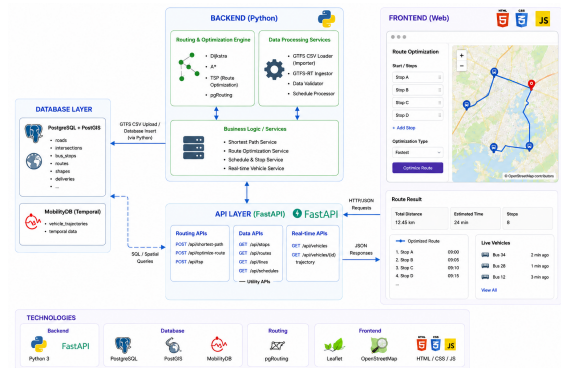


Figure 1 System architecture

The project uses FastAPI for backend services, PostgreSQL as the main database, PostGIS for spatial queries, pgRouting for network routing, MobilityDB for trajectory queries, and JavaScript/HTML/CSS for the map-based user interface.

III. DATA PREPARATION AND DATABASE DESIGN

The GTFS dataset was imported from CSV files into PostgreSQL tables. The imported dataset contains 8 agencies, 499 routes, 7,073 stops, 14,389 trips, 199,980 stop-time records, and 149,845 shape points. As shown in Table 1, the imported GTFS dataset contains the main

Table 1 Imported GTFS Dataset Summary

Table	Records	Purpose
agency	8	Agencies
routes	499	Routes
stops	7,073	Stop locations
trips	14,389	Scheduled trips
stop_times	199,980	Times
shapes	149,845	Shape points
calendar	49	Service dates

public transportation tables required for routing and spatial analysis.

During data import, different file encodings were handled because some Turkish characters caused reading errors. The import script first tries UTF-8 and then CP1254. After the relational data was loaded, additional geometry columns and routing tables were created.

IV. SPATIAL DATA TRANSFORMATION

After importing the relational GTFS data, the next step was to transform stop and route information into spatial data. Stops were converted into point geometries using their longitude and latitude values. This made it possible to show stops on a map and perform distance-based queries.

```
ALTER TABLE stops
ADD COLUMN geom geometry(Point, 4326);

UPDATE stops
SET geom = ST_SetSRID(
  ST_MakePoint(stop_lon, stop_lat),
  4326
);
```

Listing 1 Creating geometry points for stops

In this SQL statement, `ST_MakePoint` creates a point from longitude and latitude values, while `ST_SetSRID` assigns the WGS84 coordinate system. This allows the database to understand the stop locations as real geographic points.

Route shapes were also converted into line geometries. The points belonging to each shape were ordered by their sequence numbers and connected into lines. These line geometries represent the physical paths of public transportation routes.

```
CREATE TABLE shape_geoms AS
SELECT
  shape_id,
  ST_MakeLine(
    geom ORDER BY shape_pt_sequence
  ) AS geom
FROM (
  SELECT
    shape_id,
    shape_pt_sequence,
    ST_SetSRID(
      ST_MakePoint(shape_pt_lon, shape_pt_lat),
      4326
    ) AS geom
  FROM shapes
) AS ordered_shapes
GROUP BY shape_id;
```

Listing 2 Creating route shape geometries

V. ROUTING IMPLEMENTATION

To perform routing, a transit edge table was created from consecutive stops in the same trip. Each edge represents movement from one stop to the next stop. The cost of each edge is calculated based on the travel time between the two stops.

```
CREATE TABLE transit_edges_raw AS
SELECT
  st1.stop_id AS from_stop_id,
  st2.stop_id AS to_stop_id,
  GREATEST(
    gtfs_time_to_seconds(st2.arrival_time) -
    gtfs_time_to_seconds(st1.departure_time),
    1
  ) AS travel_seconds
FROM stop_times st1
JOIN stop_times st2
ON st1.trip_id = st2.trip_id
AND st2.stop_sequence = st1.stop_sequence + 1;
```

Listing 3 Creating transit edges

This table is important because routing algorithms need a graph structure. In this graph, stops are treated as nodes, and the connections between consecutive stops are treated as edges. The travel time is used as the edge cost.

The shortest path query was implemented using `pgrRouting`. Dijkstra's algorithm was used to find the minimum-cost path between two selected stops.

```
SELECT *
FROM pgr_dijkstra(
  'SELECT id, source, target, cost
   FROM transit_edges',
  start_vertex,
  end_vertex,
  directed := true
);
```

Listing 4 Dijkstra shortest path query

This query returns the ordered path between the start and destination stops. It includes the path sequence, node information, edge information, and total accumulated cost.

VI. SPATIO-TEMPORAL COMPONENT

The spatio-temporal part of the project was prepared using `MobilityDB`. The implementation follows the main concepts introduced in the `MobilityDB` Workshop, particularly the representation of moving objects as time-dependent trajectories [3]. Since real-time GTFS data for Istanbul was not available in the first phase, sample vehicle movement records were created. These records include vehicle ID, route ID, latitude, longitude, speed, heading, and timestamp.

`MobilityDB` enables representing a vehicle's movement as a trajectory rather than as separate location points. This allows queries such as finding a vehicle's position at a specific time, displaying the path it followed, and clipping a trajectory within a selected region.

For example, a vehicle trajectory can be queried based on temporal or spatial boundaries. This part of the project is useful because it prepares the system for future real-time GTFS integration.

VII. WEB INTERFACE AND API

The backend of the system is implemented using FastAPI. The API provides endpoints for accessing GTFS data, stops, routes, routing results, analysis results, and spatio-temporal vehicle data. The web interface communicates with these endpoints and displays the returned data on a map.

Some example system functions are:

- Listing all public transportation routes
- Searching and displaying stops
- Finding nearby stops
- Displaying route shapes on the map
- Calculating shortest paths between stops
- Showing busiest stops according to selected time ranges
- Displaying sample vehicle trajectories

The web interface helps users interact with the database visually instead of writing SQL queries manually. This makes the system easier to use for transportation analysis. The map visualization part was prepared using Leaflet and OpenStreetMap, which allows stops, route shapes, and routing results to be displayed interactively.

VIII. RESULTS AND DISCUSSION

The project successfully imports scheduled GTFS data, stores it in a spatial database, and provides map-based query results. Stops can be displayed as points, route shapes as lines, and shortest paths can be calculated using the generated transit network. The database also supports analytical queries such as route inspection and busiest-stop detection.

The main limitation of the current system is the lack of real-time GTFS data for Istanbul. As a result, the spatio-temporal component is currently demonstrated using sample vehicle movement data. Another limitation is that the routing graph is based on consecutive GTFS stops rather than a full road network. This is suitable for transit-based routing, but future versions can improve accuracy by integrating detailed road network data. The system was tested through API endpoint checks, database validation, routing result verification, frontend testing, and basic error handling tests.

IX. CONCLUSION

This project demonstrates how GTFS data can be transformed into a spatial and spatio-temporal query portal using PostgreSQL, PostGIS, pgRouting, MobilityDB, and FastAPI. The system provides a useful basis for public transportation visualization, shortest-path analysis, route inspection, and time-dependent vehicle movement queries.

In future work, real-time GTFS data can be integrated into the system. Query performance can also be improved through additional indexing, optimized routing tables, and a larger set of real-world public transportation test cases.

REFERENCES

- [1] B40 Cities / IETT, "Iett gtfs data," <https://opendata.b40cities.org/dataset/iett-gtfs-data>, 2023, accessed: 2026-06-13.
- [2] GeeksforGeeks, "System design vs software design," <https://www.geeksforgeeks.org/system-design/system-design-vs-software-design/#what-is-software-design>, accessed: 2026-06-13.
- [3] M. Sakr, E. Zimanyi, and M. Team, "Mobilitydb workshop," <https://docs.mobilitydb.com/MobilityDB-workshop/master/index.html>, 2022, accessed: 2026-06-13.